NumPy and PyTorch Foundations

1 Row-wise distances

For $X, Y \in \mathbb{R}^{n \times d}$, each row is one vector, so the desired result is

$$D_{i,1} = \sqrt{\sum_{j=1}^{d} (X_{ij} - Y_{ij})^2}, \qquad D \in \mathbb{R}^{n \times 1}.$$

The reduction must occur along the feature axis, while keeping a singleton dimension for the result. Summing every entry instead produces one scalar.

```
diff = x - y                          # (n, d)
summed = np.sum(diff**2, axis=1, keepdims=True)
distances = np.sqrt(summed)           # (n, 1)
```

For example, $X = [(0, 0), (3, 4)]$ and $Y = [(3, 4), (3, 0)]$ give $D = [(5), (4)]$. Keeping the result two-dimensional prevents accidental broadcasting of an $(n,)$ array against an $(n, 1)$ target into an (n, n) array.

2 Assembling four squares

Concatenate along columns to construct each row of squares, then along rows to combine the two rows:

```
top = torch.cat((square_1, square_2), dim=1)
bottom = torch.cat((square_3, square_4), dim=1)
combined = torch.cat((top, bottom), dim=0)
```

Each input is (n, n), each intermediate is $(n, 2n)$, and the result is $(2n, 2n)$. The default concatenation dimension is 0, which would stack all four inputs vertically rather than construct the intended grid.

3 From video frames to a filmstrip

Stack N frames with shape (H, W, C) into (N, H, W, C). To join frames horizontally without interleaving their rows, first permute to (H, N, W, C):

```
frames = torch.as_tensor(np.stack(video_frames, axis=0))
n, h, w, c = frames.shape
filmstrip = frames.permute(1, 0, 2, 3).reshape(h, n*w, c)
```

The pixel mapping is

$$\text{filmstrip}[h, nW + w, c] = \text{frames}[n, h, w, c].$$

A direct reshape of (N, H, W, C) to (H, NW, C) does not generally satisfy this identity. The input list is assumed nonempty with equal frame shapes.

4 Linear regression and optimization

For a batch $X \in \mathbb{R}^{B\times d}$ and targets $Y \in \mathbb{R}^{B\times m}$, a linear layer computes $\widehat{Y} = XW^\top + b$ with $W \in \mathbb{R}^{m\times d}$. Train it by minimizing mean squared error:

$$L(W, b) = \frac{1}{Bm}\sum_{i=1}^{B}\sum_{j=1}^{m}(\widehat{Y}_{ij} - Y_{ij})^2.$$

```
def forward(self, x):
    return self.linear(x)

loss_fn = torch.nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
for epoch in range(num_epochs):
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
```

Clearing gradients prevents accumulation across updates. Backpropagation computes parameter gradients; the optimizer then changes the parameters. Targets and predictions must have the same shape. In the scalar-output case, both should be $(B, 1)$.

4.1 Checkpoint selection

The exercise provides training and test data, without a separate validation set. Therefore the reference implementation selects the checkpoint by *training* loss and reserves the test set for final evaluation. In a larger experiment, introduce a validation set for model selection.

A saved parameter state must correspond to the loss used to select it. The reference saves a new best state before the optimizer update and also evaluates the final update. Calling `torch.save` immediately writes the state to disk; holding `state_dict()` in memory for later use would require copying the tensors to preserve that earlier state.

```
state = torch.load(path, map_location="cpu", weights_only=True)
out_dim, in_dim = state["linear.weight"].shape
model = LinearRegression(in_dim, out_dim)
model.load_state_dict(state)
model.eval()
with torch.no_grad():
    predictions = model(test_x)
```

5 Checks that reveal real mistakes

Verify distances on unequal rows, concatenate squares with distinct values, and construct a filmstrip with multiple rows and frames so ordering errors are visible. For regression, check that loss decreases, predictions retain their target shape, and checkpoint loading reproduces the saved model's predictions. A deterministic linear target is useful for a small smoke test; it does not establish performance on arbitrary datasets.

Solution scope. The array identities are exact under the stated shape assumptions. Regression convergence depends on the data, optimization settings, and training duration; no universal convergence claim is made.